

WP-4 Core Booking Engine

Weeks: Weeks 3–4 Track: Backend

Goal. The heart of the product: creating bookings that respect the rules and, critically, never collide. Build it correctly for one user first (Week 3), then make it bulletproof under concurrency (Week 4).

Tasks — Week 3 (correct for a single user)

- ☐ Create a one-off booking for an available slot.
- ☐ Reject bookings outside availability, inside blackout, or over capacity.
- ☐ Return a clear, machine-readable reason on rejection.
- ☐ Let a member view and cancel their own bookings.
- ☐ Write tests for the single-user happy path and each rejection reason.

Tasks — Week 4 (correct under concurrency)

- ☐ Write a test that fires two bookings for the same slot simultaneously and watch it fail.
- ☐ Choose and implement a concurrency strategy that makes a double-booking impossible.
- ☐ Prove the fix with a concurrent test that now passes.
- ☐ Document which strategy you chose and why.

HARD PROBLEM Concurrent booking collisions. “Check if free, then save” quietly fails when two requests arrive at once. SQL Server has no exclusion constraint to lean on, so the guarantee has to live inside the transaction — an application lock (sp_getapplock) keyed on the resource, or SERIALIZABLE isolation with a covering index and retry on deadlock. A filtered unique index only works for the simple one-unit-per-slot case, not overlapping time ranges. Pick a strategy and defend it. “It’s unlikely” is not an answer.

OPEN DECISION Can a TenantAdmin cancel another user’s booking, and if so, how is that user notified? Decide, document, and raise it.

Acceptance criteria

- ✓ Given one remaining slot and two simultaneous requests, exactly one succeeds and the other gets a clear rejection — never both.
- ✓ All rule violations (availability, blackout, capacity) are rejected with clear reasons.
- ✓ A member can cancel their own booking; the slot is freed.
- ✓ The concurrency strategy is documented and defended.

WP-5 Recurrence, Approvals & Time Correctness

Weeks: Week 4 Track: Backend

Goal. The features that separate a real platform from a toy: repeating bookings, approval chains, and time that behaves correctly across zones and DST.

Tasks

- ☐ Create recurring bookings (daily/weekly/monthly) with interval and end condition.
- ☐ Make each occurrence independently viewable and cancellable.
- ☐ Support cancelling one occurrence or the whole remaining series.
- ☐ Surface collisions/blackout conflicts at creation — never drop them silently.
- ☐ Implement the approval workflow: Pending → approve/reject → notify; re-check availability at approval time.

- ☐ Store all times as UTC; render in the correct local zone.
- ☐ Define and implement DST-transition behavior for recurring bookings.

HARD PROBLEM Recurrence — store the rule or materialize every occurrence? Both are defensible; justify your choice against a two-year series.

HARD PROBLEM Timezone & DST — a recurring 02:30 booking on a spring-forward day either doesn't exist or exists twice. Define the behavior, implement it, and make sure it neither crashes nor silently duplicates.

OPEN DECISION When a recurring series overlaps a newly-added blackout period, what happens to the affected occurrences? And: is availability expressed in the resource's timezone or the booker's? Decide and document both.

Acceptance criteria

- ✓ A recurring series is created, and single occurrences and the whole series can each be cancelled.
- ✓ Conflicting occurrences are surfaced at creation.
- ✓ Approval re-checks availability, so approving a since-taken slot fails safely.
- ✓ The DST edge case resolves per your documented policy with no crash or silent duplicate.